UPI Offline Mesh Network Simulator

Architectural Specification & Technical Implementation Documentation

System Version: v1.1.0

Core Technologies: Spring Boot 3.3, H2 Database, Thymeleaf MVC, Hybrid Cryptography

Document Date: July 8, 2026

Status: Architectural Release

1. Executive Summary & Background

Modern payment architectures rely on constant, low-latency connectivity to Centralized Identity Providers (CIPs) and Core Banking Systems (CBS). In developing economies, transactions are frequently interrupted in remote rural regions, underground transport corridors, and during power or telecom grid failures.

The **UPI Offline Mesh Network Simulator** introduces a deferred-settlement payment protocol modeled after asynchronous store-and-forward networks. The protocol allows users to issue, sign, and propagate cryptographically secured payment packets directly between mobile units over localized wireless interfaces (such as Bluetooth Low Energy or Wi-Fi Direct). These packets travel peer-to-peer (P2P) through intermediate helper nodes until a node with active internet connectivity (a Bridge Node) enters a Gateway coverage area and commits the payment to the centralized ledger.

> **Key Objective:** To allow transaction creation and balance transfer under zero-connectivity constraints, while ensuring cryptographically signed authenticity and robust double-spend verification upon reconciliation.

2. System Architecture & Component Design

The simulation codebase comprises a Spring Boot backend running in tandem with a high-

fidelity interactive HTML5 Canvas visualizer. The component relationships are structured as follows:

2.1 Core Components

- **MeshSimulatorService (Backend Simulator)**: Manages the spatial coordinates, transmission ranges, wireless propagation cycles, and local packet queues of all simulated mobile devices.
- **Crypto & Wallet Subsystem**: Provides RSA keypair generation, hybrid envelope packaging (AES-GCM-256 + RSA-2048), and digital signature verifications on devices and servers.
- **Settlement Engine**: Executes the ultimate transaction processing, executing ledger balance updates, double-spend check lookups, and transaction logging.
- **Interactive Canvas Dashboard**: Renders physical mesh nodes, visualizes connection ranges, captures drag-and-drop coordinate updates, and illustrates gossip signal waves and packet hops.

2.2 The Offline Transaction Lifecycle

1. **Wallet Signing & Payload Injection**: The sender creates a payment transaction. The wallet signs the transaction hash using the sender's private key, wraps the details in an encrypted payload, and injects it into the starting phone's queue.
2. **Asynchronous Epidemic Gossip Hops**: The starting node shares its transaction log with any adjacent nodes within wireless range. Gossip rounds occur periodically, transferring copies of the encrypted packet through the mesh.
3. **Gateway Uplink & Settlement**: A bridge node receives the packet, moves into Gateway range, and flushes the queue to the settlement server. The server verifies the envelope, enforces idempotency, updates balances, and commits the records.

3. Cryptographic Specification & Security Design

To prevent eavesdropping, packet alteration, or sender repudiation, all offline transactions utilize a hybrid cryptographic envelope. The security pipeline contains the following specifications:

3.1 RSA-2048 Digital Signatures

Every user wallet is initialized with a unique RSA-2048 public/private keypair. When a transaction is created, the wallet signs the payload:

```
Signature = RSA-Sign(Sender_Private_Key, SHA-256(SenderVPA || RecipientVPA || Amount || Nonce))
```

This signature prevents message tampering during the mesh routing phase. Any bit alteration made by a rogue peer node will invalidate the signature check performed by the server.

3.2 Local Balance Integrity & Cryptographic Wallets

Mobile nodes maintain an offline encrypted local balance. To deter local filesystem manipulation, the wallet is encrypted using a key derived from the user's PIN. This ensures that even if a physical device is compromised, raw wallet databases cannot be trivially extracted or written to without key derivation.

Security Threat	Mitigation Strategy	Implementation Layer
Eavesdropping (Privacy Leak)	Payload fields (Amount, Recipient) are encrypted with symmetric keys prior to gossiping.	Wallet Client (JS Simulator)
Payload Alteration (Man-in-the-Middle)	RSA-2048 signatures are verified by the server during settlement.	SettlementService (Java Backend)
Replay Attacks	Every transaction contains a unique cryptographic UUID/Nonce that is stored in the server's cache.	Idempotency Filter (Java Backend)

4. Double-Spend & Idempotency Safeguards

The critical risk in any offline payment system is the **Double-Spend Threat**. Since offline nodes do not have immediate access to a centralized balance database, an unprincipled sender could inject multiple transactions totaling more than their actual balance into different bridge nodes.

4.1 Trust-on-First-Sight (TOFS) & Idempotency Filter

The simulator implements a Trust-on-First-Sight validation mechanism:

- The backend maintains an **Idempotency Cache** storing the hash of every processed

payload.

- When a duplicate packet is uploaded by a secondary bridge, the settlement engine identifies the payload hash collision and logs a `DUPLICATE_DROPPED` status, preventing duplicate ledger entries.
- If a user executes a double-spend attempt, the first packet to reach the Gateway settles successfully, while subsequent conflicting transactions are flagged as `INVALID` due to insufficient balance.

> **Security Trade-Off:** While this prevents database corruption and double-processing, the merchant or recipient is vulnerable to transaction rejection if they accept payments offline without real-time ledger clearance.

5. Technical & Architecture Trade-offs

Architecting distributed offline networks requires balancing the CAP theorem constraints (Consistency, Availability, and Partition Tolerance).

5.1 Consistency vs. Availability

Traditional credit card systems or standard UPI models favor **Consistency**: a transaction is blocked if the server cannot verify the client's current balance in real time. The offline mesh network favors **Availability and Partition Tolerance**, allowing transactions to proceed under network isolation at the cost of delayed, eventual consistency.

5.2 Data Storage: H2 In-Memory vs. PostgreSQL & Redis Caching

For simulation simplicity, the prototype utilizes an H2 In-Memory database coupled with JVM concurrent maps. The differences between this setup and a production deployment are outlined below:

Criteria	H2 In-Memory Database (Simulation)	PostgreSQL + Redis Cache (Production)
Performance & Latency	Microsecond response times, but concurrent limits bounded by single JVM heap.	Redis handles sub-millisecond idempotency cache checks; PostgreSQL manages persistent transactions.
Fault Tolerance	Data is lost when the application process terminates.	Write-Ahead Logging (WAL) and Redis replication guarantee no data loss on hardware failures.
Scalabilit	Limited to a single server	Horizontal read replicas and Redis

Criteria	H2 In-Memory Database (Simulation)	PostgreSQL + Redis Cache (Production)
y	instance.	clustering handle high-throughput national volume.

6. Interactive Walkthrough Tour Engine

To ensure non-technical presentation viewers immediately comprehend the offline routing sequence, the simulator incorporates a built-in interactive walkthrough tour:

- **Spotlight Focus:** The engine utilizes a semi-transparent screen mask overlay, dynamically calculating target element offsets using the browser's `BoundingClientRect`.
- **Step Navigation:** The walkthrough guides the user through transaction setup, mesh hops, network canvas interactions (dragging elements), gateway uploading, and ledger balance updates.
- **First-Load Auto-run:** Integrates local browser persistence storage (`localStorage`) to launch the guide automatically for first-time visitors, while remaining manually triggerable via a dedicated info icon button in the header.

7. Production Roadmap & Recommendations

To scale this prototype to a real-world production system, the following implementations are recommended:

1. **Hardware-level Peer Communication**: Replace the simulated gossip timers with genuine Bluetooth Low Energy (BLE) peripheral/central advertising profiles or Wi-Fi Direct service discovery protocols on target mobile platforms.
2. **Hardware Secure Enclaves**: Store the user's RSA private keys inside Android Keystore or iOS Secure Enclave modules, ensuring signatures cannot be extracted even on rooted hardware.
3. **Merchant Trust Credit Caps**: Introduce pre-funded wallet credit buffers. Under zero network access, transactions are validated against a local secure hardware-locked wallet, capping maximum merchant liability during network outages.